WPT Day 9 Evening

Abhishek Purohit · Thu 29 Oct 26 · Securing the CRUD APIs: admin registration with bcrypt-hashed passwords, the login API, JSON Web Tokens (header / payload / signature, secret keys), the `Authorization: Bearer` request header, and a `verifyToken` middleware that guards the product APIs

---

Table of Contents

1. Why the CRUD APIs Must Be Secured
2. The Register, Login, Token, Call Flow
3. Never Store Plain-Text Passwords
4. bcrypt: hash vs hashSync
5. Hashing the Password in the Register API
6. The Login API: Design
7. The Login API: Implementation and Testing
8. What a JWT Is: Header, Payload, Signature
9. Why the Signature and Secret Key Matter
10. Secret Keys in Real Applications
11. Generating the Token with jsonwebtoken sign
12. Where to Send the Token: URL vs Body vs Header
13. The Authorization Header and the Bearer Format
14. The verifyToken Middleware: Header Check
15. The verifyToken Middleware: jwt.verify
16. Guarding the Product Routes and End-to-End Test
17. Beyond Basic Authentication
18. Combined Worked Example: Secured CRUD Server
19. Quick Revision Checklist
20. Practice Exercises
21. Practice Answers

Why the CRUD APIs Must Be Secured

The problem

By the end of Day 8 you have an Express + MySQL application with product APIs: **create** a product, **update** it, **fetch** one or all, **delete** one. The morning session of Day 9 added an `admin` entity (and an API to register an admin).

But look at what the APIs allow today:

- Anyone who learns the URL of the create / update / delete API can call it.

- It does not matter whether the caller is an admin, a normal user, or a stranger who has never registered with your application.
- Postman, a script, a browser — the server answers all of them the same way.

The lecture's wording: *"I don't want people who are just directly getting the URL to be able to access your API. Currently whenever anyone has your API URL for creating the product, deleting, updating — all those users can call your API. I don't want that to happen anymore."*

What we want instead

Only a **logged-in admin** may run the product operations. That sentence contains three separate jobs:

Job	Question it answers	API involved
Registration	Who is allowed to exist as an admin?	`POST /admin`
Login (authentication)	Is this person really that admin?	`POST /admin/login`
Access control	Is this request coming from a logged-in admin?	every secured API (product APIs)

Background: **Authentication** means proving who you are (login). **Authorization** means deciding what an authenticated caller may do. In this session the product APIs simply demand "must be logged in"; later in the course (and in real systems) authorization grows into roles such as admin vs customer.

Session setup (what the lecturer did)

1. Copied the previous project folder (`crud-apis`) into a new folder for this session and added a notes file.
2. Opened it in VS Code. There was no `node_modules`, so from **inside the project folder** he ran `npm install` (he first ran it from the wrong folder — always `cd` into the folder that contains `package.json`). This installs Express and mysql2.
3. Observed that the existing APIs (`productRouter` , `welcomeRouter`) are all **open APIs**.

> **Exam:** An *open API* can be called by anyone who knows its URL. A *secured API* demands proof of login (a token) before doing its work.

The Register, Login, Token, Call Flow

The whole story in one picture

Think about it from first principles. Before an admin can *log in*, the admin must *exist* in the database — "if there is no data for the admin, who is going to log in?" Once logged in, the server must give the admin some **proof of the successful login**; otherwise every later request would have to send the username and password again. That proof is a **token**.

```
STEP 1  REGISTER            POST /admin
        client → { name, phone, password } → server
        server hashes the password → INSERT into admin table

STEP 2  LOGIN               POST /admin/login
        client → { phone, password } → server
        server checks phone + password against the DB
        server creates a TOKEN (JWT) → sends it in the response

STEP 3  CALL SECURED APIs   GET/POST/PUT/DELETE /product ...
        client → request + Authorization: Bearer <token>
        verifyToken middleware checks the token
            missing  → 401 "token is missing"
            invalid  → 401 "token is not valid"
            valid    → next() → the real controller runs
```

Key rules from the lecture

- **Register** and **login** APIs stay **open** — "Login API ko secure mat karna; login karne ke liye login karna zaroori nahi hai." Anyone must be able to reach them, or nobody could ever get in.
- Every API that handles a protected resource (the product APIs) demands the token.
- The token is the **only proof** of a successful login: *"login successful hua ya nahi hua uss cheez ka proof kaun hoga? That is only the token."*
- If the request has a token → the user is logged in → the work proceeds. If it has none → reject immediately with a response such as "token is missing", "please log in first", or "user is not authenticated".

Understanding beats implementation

The lecturer paused on login behaviour and asked why *understanding* matters more than typing the code. A student answered: AI will do the implementation. His reply: AI is there to take the coding job, so *"only orchestrators are going to save their job — you have to become knowledgeable."* The takeaway for study: be able to explain the flow above without looking at code.

> **Exam:** Flow = register → login (server issues token) → client sends token on every secured call → middleware verifies → controller runs. Register and login are never secured.

Never Store Plain-Text Passwords

The loophole found in the morning session

In the morning the register-admin API inserted the password **exactly as typed**. So the `admin` table contained rows like `Suresh123` in the `password` column.

Why that is unacceptable:

- Every database developer or administrator with table access can read every user's password.
- If the database leaks, every account is instantly compromised.
- Many users reuse one password across sites, so one leak hurts them elsewhere too.

Rule from the lecture: *"Never ever try to store passwords as plain text — or any secret data as plain text — inside your database."*

What we want from the "encryption"

The lecturer says "encrypt", and uses the word loosely throughout the session. His requirement is precise, though: **once the password is encrypted, nobody should be able to decrypt it again.**

His analogy for why reversible encryption is pointless: if anyone can easily decrypt the password, it is like *"Mithun hiding behind a bicycle to dodge bullets"* — you look protected but you are not.

Background: The technical name for what he wants is **hashing** — a one-way transformation.

	Encryption (reversible)	Hashing (one-way)
Can you get the original back?	Yes, with the key	No
Used for	Data you must read later (messages, files)	Passwords, which you only ever need to *compare*
Lecture's term	"encrypt"	"encrypt such that you cannot decrypt backwards"

If the original can never be recovered, how do we check a login attempt? We hash the attempt the same way and compare — or let the library do it (the `compare` function, covered later in this session).

Algorithms and why we use a library

There are many password-hashing algorithms (the lecturer mentioned **SHA-256** and **bcrypt**, names you may have met at school or college). We will not write the algorithm ourselves:

1. We do not know how to.
2. There is no time for that effort.

So we use a **third-party module** that provides a tested implementation "out of the box". The module is `bcrypt`. You give it the plain-text password; it gives back the hashed string.

> **Exam:** Passwords must be stored hashed (one-way). Never plain text. The lecture uses the `bcrypt` package, "plain password in, encrypted password out".

---

bcrypt: hash vs hashSync

Installing

From the project folder:

```
npm install bcrypt
```

The lecture installed version 6. It is added to the `dependencies` of `package.json`.

The two functions

The `bcrypt` module exposes a pair of functions that do the same job with different execution styles:

Function	Style	What happens to the next lines of code?
`hash`	**Asynchronous**	Does not make the rest of the code wait; later lines run while hashing is in progress
`hashSync`	**Synchronous**	The code waits until the hash is ready, then continues

The lecture's reasoning for choosing `hashSync` in the register API: *the data must be inserted only once the password has been encrypted.* If you used `hash` without waiting, the INSERT line could run before the hash exists. So we want synchronous behaviour — hence the `hashSync` variant.

```
import { hashSync } from "bcrypt";
```

(The lecturer noted that `hash` is imported from the `bcrypt` module in exactly the same way: `import { hash } from "bcrypt"`.)

Background: `hash` can also be used with `await` (it returns a Promise when you give no callback), which keeps the server responsive while hashing. `hashSync` blocks Node's single thread while it computes — the same CPU-bound vs I/O-bound idea from Day 7. For a small learning project `hashSync` is perfectly fine and simplest, which is why the lecture uses it.

Arguments and return value

```
const encryptedPassword = hashSync(password, 10);
```

Argument	Meaning
1st	The **plain-text password string**
2nd	Either a **key (salt)** on the basis of which the password is encrypted, **or the number of rounds** — "how many times to shuffle before the final value"

The lecturer's simplification: pass the plain password and pass `10` rounds; it encrypts 10 times and returns the final value. The **return value is a string** representing the encrypted password.

Background: In bcrypt the number is called the **salt rounds** (cost factor). It sets how much work each hash takes — each +1 doubles the work — which deliberately slows down brute-force guessing. 10 is a common default. Internally bcrypt also generates a random **salt** per hash.

> **Exam:** `hashSync(plainPassword, rounds)` → returns the hashed string. `hash` = async, `hashSync` = sync. We pick sync so the INSERT happens only after hashing finishes.

Hashing the Password in the Register API

The change to the controller

In `AdminController.js`, the `save` function reads `name`, `phone`, `password` from `request.body`. Before inserting:

1. Hash the plain password into a variable (the lecture names it `encryptedPassword`).
2. In the `values` array of the INSERT, pass `encryptedPassword` — **not** `password`.

```
import { hashSync } from "bcrypt";
import { sqlConnectionPool } from "../configs/Dbconfig.js";
```

```
export async function save(request, response) {
  try {
    const { name, phone, password } = request.body;

    const encryptedPassword = hashSync(password, 10);

    const insertQuery =
      "INSERT INTO admin (name, phone, password) VALUES (?, ?, ?)";
    await sqlConnectionPool.query(insertQuery, [name, phone, encryptedPassword]);

    response.send({ message: "admin got registered" });
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
}
```

Guarantee: the database **never** stores the plain-text password; only the encrypted value goes in.

Demo in Postman

- Run `node index.js`.
- **POST** `/admin`, Body → raw → JSON:

```
{
  "name": "Suresh",
  "phone": "808080",
  "password": "Suresh123"
}
```

- Response: *admin got registered*.
- In the database run `SELECT * FROM admin;` — the `password` column now holds a long encrypted-looking string, not `Suresh123`.

Widening the column

The lecturer ran `DESCRIBE admin` and saw the password column length was **500**. To be sure a larger encrypted value would never be truncated or rejected he widened it. He had forgotten the exact command — a student reminded him the keyword is `MODIFY` (not `UPDATE` or `CHANGE`):

```
ALTER TABLE admin MODIFY password VARCHAR(1000);
DESCRIBE admin;
```

Background: A typical bcrypt hash is about 60 characters, so 500 would already have sufficed; the lecture simply chose a safe, large limit.

Same password, different hash

He registered a second admin — **Nitin**, phone `909090`, password `Nitin123` — and queried again. Important rule:

> If two users have the **same** password, their encrypted values are still **totally different**.

So never assume that two identical hashes mean identical passwords (and never assume you can spot a duplicate password from the table).

Background: This is the random **salt** doing its work. It also explains why login cannot be "hash the typed password and use SQL `=`": hashing the same text twice yields two different strings. Only bcrypt's `compare` function knows how to check a plain password against a stored hash.

Small lecture hiccup

A Postman response seemed unchanged after a change — the lecturer guessed caching and re-sent the request in a fresh tab. If a response looks stale during practice, **restart the server** and send the request again.

> **Exam:** Register API → `hashSync(password, 10)` → INSERT `[name, phone, encryptedPassword]`. Same plain password gives different hashes. Widen the password column with `ALTER TABLE admin MODIFY password VARCHAR(1000)`.

The Login API: Design

What login really does

Open Facebook or Flipkart and press Login. The page calls a **login API** with your username (or email, or phone number) and password. The server **compares those two values with the actual database values**:

- Values match → login successful.
- Values do not match → login fails.

In this lecture the identifying field is **phone number** (it could be email or username in another app).

Creating the controller function

In `AdminController.js` add a second exported `async` function named `login` with the usual `(request, response)` arguments. Copy the `try / catch` pattern from `save` — every API keeps the try/catch with a 500 "something went wrong".

On the frontend the login page has a **form with two inputs**; the frontend sends them in the **request body**. So the login API reads:

```
const { phone, password } = request.body;
```

Why the password cannot be compared in SQL

The lecturer asked the class a series of questions.

Q: What will the login request carry — encrypted or plain password? A: **Plain.** The client is the end user. It knows nothing about the encryption done by the server.

Q: Can you compare that plain-text password directly in the SQL query against the database column? A: **No.** The database holds the *encrypted* value; the request holds the *plain* value. They can never be equal. *(A student: "it was already encrypted when saved in the database.")*

So the plan has two stages:

1. **Find the admin by phone number** with SQL (equality on phone works because the phone was stored as-is).
2. **Compare the password in JavaScript**, using bcrypt.

```
request body: { phone, password(plain) }
       |
       v
SELECT * FROM admin WHERE phone = ?      ← SQL compares PHONE only
       |
  rows empty? --yes──> "phone number is not valid"
       |
       no  (admin found; row holds the encrypted password)
       v
compareSync(plainPassword, adminData.password)   ← bcrypt compares PASSWORD
       |
  true  --> login successful (and token)
  false --> "password is not valid"
```

> **Exam:** Never compare plain password with a stored hash in SQL. Step 1: SELECT by phone. Step 2: bcrypt compares plain vs hash in JS.

The Login API: Implementation and Testing

Step 1: query by phone (still parameterised)

```
const loginQuery = "SELECT * FROM admin WHERE phone = ?";
const [result] = await sqlConnectionPool.query(loginQuery, [phone]);
```

- Use `?` and a values array instead of concatenating the phone — **to avoid SQL injection** (the Day 8 rule).
- `query` returns a Promise, so `await` it, which forces the function to be `async`.
- Destructure the returned array to take the result rows (`[result]`).

Step 2: add the route first, with a dummy response

For the first test the lecturer logged `result` to the server console and sent a dummy `response.send({ message: "login process" })`. He then registered the route in the admin router:

```
adminRouter.post("/admin/login", login);
```

`index.js` already had `adminRouter` registered, so after restarting the server the URL is `/admin/login`.

Postman: **POST** `/admin/login`, body raw JSON with `phone` and `password` (the password was not being checked yet, but he passed one anyway "so that you have an idea what request body to send to the login API").

What the result looks like

A SELECT returns an **array**:

Situation	`result`
A row matches the phone	`[ { id, name, phone, password(encrypted) } ]`
No row matches	`[]` — a blank array

So: **array has data → phone matched; array blank → phone did not match.**

Step 3: empty array → early response

```
if (result.length == 0) {
  response.status(401).send({ message: "phone number is not valid" });
```

```
  return;
}
```

Status code is the developer's choice: the lecturer said you may use **404**, an *unauthenticated* code, or *bad request* — "it is totally up to you". (Early return stops the function after sending the response.)

Step 4: pull out the admin and compare passwords

Phone numbers should be **unique** per admin. The lecturer had not made the column unique, but said: assume it is — then there is always at most one object, and `result[0]` gives you the admin's data.

```
const adminData = result[0];
```

`adminData.password` is the encrypted password; `adminData.id` is the admin's id (we will need it for the token).

Who should compare? The lecturer's question: *who encrypted the password?* **bcrypt.** Then bcrypt must also do the comparison — *"the one who encrypted is responsible for comparing, because only it knows how it encrypted."* bcrypt gives a pair:

Function	Style
`compare`	async
`compareSync`	sync

```
import { hashSync, compareSync } from "bcrypt";

const isPasswordValid = compareSync(password, adminData.password);
```

Argument	Source
1st	**plain** password — from `request.body`
2nd	**encrypted** password — from the database row (`adminData.password`)

It returns a **boolean**: `true` if they match, `false` if not.

Step 5: branch on the boolean

```
if (isPasswordValid) {
  response.send({ message: "login successful" });   // token comes later
} else {
  response.status(401).send({ message: "password is not valid" });
}
```

(In the lecture the `else` branch contained a simple "invalid password" message.)

Complete login logic (no token yet)

```
export async function login(request, response) {
  try {
    const { phone, password } = request.body;

    const loginQuery = "SELECT * FROM admin WHERE phone = ?";
    const [result] = await sqlConnectionPool.query(loginQuery, [phone]);

    if (result.length == 0) {
      response.status(401).send({ message: "phone number is not valid" });
      return;
    }

    const adminData = result[0];
    const isPasswordValid = compareSync(password, adminData.password);

    if (isPasswordValid) {
      response.send({ message: "login successful" });
    } else {
      response.status(401).send({ message: "password is not valid" });
    }
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
}
```

The lecturer's three test cases

#	Phone	Password	Response
1	Not in DB (invalid)	any	"phone number is not valid"
2	`808080` (Suresh, correct)	wrong	"password is not valid"
3	`808080`	`Suresh123` (correct)	"login successful"

The simple logic, as he put it: *no record for the phone → send an early response; record found → take it out, compare the password with the encrypted one; both right → success, else password not valid.*

> **Pitfall:** Do not tell attackers *which* of phone/password was wrong in a real application — "phone number is not valid" vs "password is not valid" helps someone probe for registered numbers. The lecture keeps two messages for teaching clarity.

> **Exam:** Login = SELECT by phone → if `[]` early response → `result[0]` → `compareSync(plain, hashed)` → boolean → success or "password invalid". `compareSync` argument order: plain, then encrypted.

What a JWT Is: Header, Payload, Signature

The problem

"Login successful" in the response is not enough. For later requests the server needs a way to recognise that this caller has already logged in. The admin might also be required to log in again only after a **time interval** — every 24 hours, every two days — a timeline you decide. During that period the admin carries **a token**.

Who generates it? **The server** (after a successful login). The token is sent back in the response, and the client keeps it.

JWT = JSON Web Token

The token we generate is called a **JWT**, which stands for **JSON Web Token**.

Why "JSON"? On the backend, while creating the token, you can place **JSON data inside it** — key/value pairs. Examples the lecturer gave:

- the **ID of the admin** who logged in,
- the **time of login**,
- the **expiry time** (after how long the token expires),
- anything else, e.g. the user's **role**.

Three parts, always

Every JWT has exactly three parts:

Part	Role	Who deals with it
Header	Metadata of the token — which **algorithm** is used to encrypt it	Set **automatically**
Payload	The **data** you want inside the token (admin id, login time, expiry, role...)	**You** specify it
Signature	Based on the **secret key** used to sign the token	Set **automatically** from the secret key you pass

The lecture's guidance: *"At development time you only have to worry about the payload."* You only need to **know** what header and signature are; the library manages them. You specify the payload, and the library reads it and — with the help of header and signature — turns the whole thing into one **alphanumeric string**. That string is the JWT.

What it looks like (jwt.io)

On the official site **jwt.io** the sample on the left is a very long string beginning with `ey ...`. Looking carefully, it contains **dots**:

```
eyJhbGciOi ... (header).eyJhZG1pbklk ... (payload).SflKxwRJSM ... (signature)
   part 1                 part 2                      part 3
```

- First encoded part → **header**
- Second → **payload**
- Third → **signature**

Two dots → three parts. The lecturer added an example of this shape to his notes file.

Background: The header and payload are **Base64URL-encoded JSON**, not truly secret. Anyone holding a token can decode and read its payload (you can paste a token into jwt.io and see it). What the signature guarantees is that the token **cannot be altered or forged** without the secret key. So the lecture's "encrypted token" really means "signed token" — never put passwords or other secrets into the payload.

> **Pitfall:** JWT payloads are readable by anyone who has the token. Put an id, role, and times in it, never a password.

Predefined function, like hashing

Just as bcrypt gave us a ready function for passwords, a library function will take **only the payload** (plus a secret key) and return the JWT; it "thinks about the header by itself".

> **Exam:** JWT = JSON Web Token = header . payload . signature (three dot-separated parts). Header: algorithm metadata. Payload: your JSON data (admin id, login time, expiry, role). Signature: tied to the secret key. Developer supplies only payload + secret.

Why the Signature and Secret Key Matter

The self-service restaurant analogy

The lecturer ran a **self-service restaurant** in the class:

1. You pay at the **payment counter** and receive a **token**.
2. You go to the **food counter** where he stands. You give the token; he sees it and gives you food.
3. But *"you guys are a little mischievous"*: the next day you arrive with a **duplicate token that looks exactly the same**, without paying.
4. This time he cannot be fooled, because the restaurant wrote **something with secret ink** on every genuine token, visible only under a torchlight. He shines the torch on yours — the secret mark is missing. Result: *you have a token, but it is not a valid token.*

Restaurant	JWT
Payment counter	Login API
Token	JWT
Food counter	A secured API (product API)
Staff checking the token	`verifyToken` middleware
Secret ink + torch	**Secret key** + signature verification
Duplicate / fake token	Forged or tampered JWT

The two checks at authentication time

When a request arrives at a secured API you check **two things**:

1. **Is a token present?**
2. **Is the token valid / authentic?** — i.e. was it generated with *our* secret key?

A token that merely *exists* is not enough, because lookalike, duplicate or invalid tokens can exist.

So at token creation

While creating a token you give the library **two things**:

1. the **payload**,
2. the **secret key** on the basis of which the token is encrypted/signed.

The signature part of the JWT reflects "the secret used to sign the JWT". Later, during verification, you supply the same secret key to confirm the token is genuine.

The client holds the token as its identity and authentication proof, and sends it whenever it calls a secured API; your server validates it each time.

> **Exam:** Signature = proof the token was made with the server's secret key. Create: payload + secret. Verify: token + the same secret. A token without a valid signature is a fake.

Secret Keys in Real Applications

What the lecture does (learning only)

In the class demo the secret is a simple hard-coded string: `hello1234`. The lecturer stressed that this is only for practice: *"In real-time applications you should not do this."*

What real applications do

Real systems use **RSA keys**, a **pair** of randomly generated keys:

Key	Use
Private key	**Encrypts / signs** the token (at login)
Public key	**Verifies / validates** the token (when requests arrive)

They are linked to each other. The lecturer demonstrated an *RSA private key generator* website: choose a key length such as **2048** bits, click generate key pair, and receive a random pair.

Where the keys live

- **Never in your code**, never directly in variables — otherwise your application is "vulnerable to exposing sensitive data".
- They are set in the **deployment environment** — where the application is deployed (the lecturer named **Docker and Kubernetes** as examples). The application **reads** them from that

environment.

Background: On a developer machine the usual first step is an environment variable or a `.env` file read with `process.env.SECRET_KEY`, kept out of Git. Environment configuration comes later in the course.

Rotate keys regularly

Keys must be **changed at regular intervals** — weekly or every fortnight, whatever you decide. His example: someone with deployment access copies the private and public keys, takes a bribe, and sells them to a competitor. The competitor could then access *any admin or customer of your application*. These are the most sensitive data you have.

Closing remark: "*We are still baby steps — we have only learned to walk.*" Do not assume that a hard-coded secret is what you will write at work.

> **Exam:** Practice: hard-coded secret string. Real life: RSA private key (sign) + public key (verify), stored in the deployment environment, never in code, rotated regularly.

Generating the Token with jsonwebtoken sign

Installing and importing

The third-party module is named `jsonwebtoken` ("JSON Web Token").

```
npm i jsonwebtoken
```

`package.json` now shows four dependencies: `bcrypt`, `express`, `jsonwebtoken`, `mysql2`.

The module provides (among others) two functions we use:

Function	Purpose
`sign`	**Create** a new JWT
`verify`	**Check** a token (used later in the middleware)

In `AdminController.js` import it with a reference name `jwt`:

```
import jwt from "jsonwebtoken";
```

Calling `jwt.sign`

Create the token **only when the password is valid** — inside the `if (isPasswordValid)` branch.

```
const token = jwt.sign({ adminId: adminData.id }, "hello1234");
```

Argument	Meaning
1st	**Payload** — a JSON object. We put the admin's id so that later the token can tell us *which admin* it belongs to. The value is `adminData.id` — the same id that was stored in the database, read exactly like `adminData.password`.
2nd	**Secret key** — the string on the basis of which the token is encrypted

`sign` returns the JWT string.

Sending it in the response

```
response.send({ message: "login successful", token });
```

In JavaScript `{ token }` is shorthand for `{ token: token }` — the key name is `token` and the value is the variable (discussed in the basics). So the JSON response contains a key called `token`.

The test result

- Invalid phone → "phone number is not valid".
- Valid phone + invalid password → "password is not valid".
- Valid phone + valid password → **login successful** plus a long `token` value — *"the actual proof of your login."*

The client must **store that token somewhere safely**, because later (delete / update / create / fetch a product) the **same token must travel with the request**. The lecturer explained a login step in one line: *login API issues the token; secured APIs demand it.*

Complete `login` function with token

```
export async function login(request, response) {
  try {
    const { phone, password } = request.body;

    const loginQuery = "SELECT * FROM admin WHERE phone = ?";
    const [result] = await sqlConnectionPool.query(loginQuery, [phone]);

    if (result.length === 0) {
```

```
      response.status(401).send({ message: "phone number is not valid" });
      return;
    }

    const adminData = result[0];
    const isPasswordValid = compareSync(password, adminData.password);

    if (isPasswordValid) {
      const token = jwt.sign({ adminId: adminData.id }, "hello1234");
      response.send({ message: "login successful", token });
    } else {
      response.status(401).send({ message: "password is not valid" });
    }
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
}
```

Background: `jwt.sign` accepts an optional third argument such as `{ expiresIn: "1h" }` that puts an expiry time into the payload. The lecturer deliberately left expiry out of this basic version (see *Beyond Basic Authentication*).

> **Pitfall:** The secret used in `sign` and the secret used in `verify` must be the **same** string. If they differ, every token will be rejected as invalid.

> **Exam:** `jwt.sign(payload, secretKey)` returns the token; call it only after a successful password comparison; send it as `{ token }` in the login response.

Where to Send the Token: URL vs Body vs Header

The question

Admin has the token. Product APIs now demand it. In a request, **where** do we put the token? The lecturer recalled that, back on the day request bodies and params were taught (about 12 days earlier), there were two ways to send data to the server: the **URL** and the **request body**. Then he introduced a third. Let us rule each in or out.

Option 1: URL — rejected

The token is sensitive data. Whatever goes on the URL can be seen by anyone ("exposed to the entire internet world"). Putting it there is, in his Hindi idiom, "*aa hacker mujhe maar*" — inviting

trouble. Student answer: "anyone can access it."

Option 2: Request body — rejected (two reasons)

1. **GET has no body.** Every API that uses the GET method (fetch all, fetch by id) would be impossible to authenticate.
2. **Separation of concerns.** The body should contain only data of your **business entity** (e.g. the product's name, price...). A token (or a "desired language") does not belong there.

Option 3: Request header — chosen

Responses have a **header** carrying metadata (type, size, who sent it...). *Similarly requests have a header too*, holding **metadata about the request**. Any method can carry headers, including GET, POST, PUT and DELETE.

Where	What it should carry	Verdict for token
URL	Resource identity and query values (ids)	No — exposed
Request body	Business-entity data	No — no body in GET; mixes concerns
Request header	**Metadata** about the request	**Yes**

Other metadata examples: **the language** in which the response should be returned. An application showing data in several languages should pass the desired language in the header, not the body — it is not business data.

Headers in Postman

The Postman request screen has a **Headers** tab beside Body. Even before you add anything you can see about **nine default headers** that Postman sends automatically, such as `Content-Type`, `Content-Length`, `Accept`, `Accept-Encoding`, and others. Headers are **key/value pairs**, and you can add your own.

When you build the frontend later you will set the method, body and headers **programmatically in code**; Postman only gives you a UI for the same thing.

> **Exam:** Three ways to send data to the server: URL, request body (business entity data), request header (metadata). A token goes in the **header** — never in the URL (exposed) or body (GET has none; separation of concerns).

The Authorization Header and the Bearer Format

A predefined header

You do not invent a header name for the token. The people who designed the web's request/response system defined a header for exactly this purpose: `Authorization`. The key is `Authorization`; the value is built from your token.

Your backend will always **read the Authorization header**, extract the token from it, validate it, and if it is missing, **restrict** the API.

The value format: `Bearer <token>`

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhZG1pbklkIjoxfQ.xxxxxxxx
               ^^^^^^ ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
               type   space + the token (the "credential")
```

Why a type word? There are **different kinds of authorization**, each with its own kind of credential. Names the lecturer mentioned: **Basic** (username + password), **Digest** (he recalled the name), and **Bearer**.

Rule for this course:

> If the credential is a **JWT**, the authorization type must be `Bearer`, then **one space**, then the token.

He told students not to go deep into Basic and Digest now — explore them in free time. What he calls "token" becomes, more formally, the **credential**.

The Postman exercise

Request: **DELETE** `/product/2` (any id).

1. Open the **Headers** tab.
2. Key: `Authorization`
3. Value: `Bearer` followed by the token you copied from the login response.

A student asked why create a header manually when Postman has an **Authorization** tab where you can pick *Bearer Token* and paste the token. The lecturer agreed: it does exactly the same — **that tab also sends the Authorization header ultimately**. Doing it by hand in Headers is useful so you know a request header exists and that the tab merely fills it. Postman's list of types includes Basic and Bearer (and more) — we choose **Bearer Token**.

Login is the one door that stays open

Anyone can call the login API (that is where tokens come from). Only APIs that handle protected resources — here the product APIs, which only an admin should manage — require `Authorization`. So the order is always:

1. Call login → receive token.
2. Call a secured API with `Authorization: Bearer <token>`.

> **Exam:** Header name `Authorization`; value `Bearer <space><JWT>`. JWT → type Bearer. Postman's Authorization tab with "Bearer Token" produces the same header.

The verifyToken Middleware: Header Check

The idea

Without touching the controllers, put a **middleware** (a function that runs first) in front of `save`, `getAll`, `getById`, `deleteById` (and `updateById`). The lecture called it "M1" at first. Recap of Day 8: middleware runs before the controller, and either sends the response itself (controller never runs) or calls `next()` to continue.

The `middlewares` folder already existed. The lecturer created `verifyToken.js` there.

Skeleton

A middleware is a function with **three arguments**: `request`, `response`, `next`. (Name it `verifyToken` or `validateToken` — any name is fine.) Start with the "good child" habit:

```
export function verifyToken(request, response, next) {
  try {
    // ... checks ...
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
}
```

His speech on try/catch: you should trust your code, but one day it will crash — especially **during the demo to the client**. A try/catch ensures the server still sends a proper response. `console.log(error)` is for **local debugging**; do not leave console logging in deployment or production.

Reading a header: `request.get`

You have read data three ways so far:

Data location	How to read
URL params	`request.params`
Request body	`request.body`
Request header	`request.get("headerName")` — a function call

A student guessed `request.header` / `request.headers` — wrong according to the lecture. The function is `request.get` and you pass the **header name**:

```
const authHeader = request.get("authorization");
```

(Header names are case-insensitive: `"Authorization"` works as well.)

Extracting the token

The header value is a string of the form `prefix + space + token` — always. So:

1. `.split(" ")` — split on the **space** → array of two strings: `["Bearer", "<token>"]`.
2. Take **index 1**: `[1]` → the actual JWT.

```
const token = authHeader.split(" ")[1];
```

This is "line number 6" in the lecturer's file: the highlighted `authHeader.split(" ")` returns the array, then `[1]` reads the second element. He warned that this is unusual code and to follow it carefully. The bearer prefix is not needed for validation, only the token is.

First check: does the header exist?

Do not jump to validating the token. If there is **no Authorization header**, there is nothing to validate. So structure it as:

```
const authHeader = request.get("authorization");

if (authHeader) {
  // header present: read token, verify it (next section)
} else {
  response.status(401).send({ message: "token is missing, please log in first" });
}
```

Why **401 Unauthorized**? Status codes starting with **4** mean the **client's fault**, not the server's. The caller is trying to access a resource without authorising first. The message wording is free: *token is missing*, *please log in first*, *user is not authenticated* — "it could be anything."

In the `else` path the middleware sent the response and **did not call** `next()`. So delete/update/create/get never runs.

> **Pitfall:** Forgetting that a middleware that sends a response must not also call `next()` — the controller would run and try to answer a second time.

Demo: header existence only

With just this existence check applied to the product routes:

- **GET** `/product` with **no** Authorization header → the server blocks it with *token is missing*; no product data.
- Add an Authorization header with **any random value** and send again (he used a fresh Postman request tab) → the products come back! The middleware only checks that *a* header exists; the validity is not tested yet.

The lecturer's point: this was just to show how the middleware gates the request. Real protection needs the **validity check** next.

> **Exam:** Read headers with `request.get("authorization")`. Token = `authHeader.split(" ")[1]`. No header → 401 and do not call `next()`.

The verifyToken Middleware: jwt.verify

Why a second check

The existence of a header proves nothing — a fake token passes. We must verify the token's **authenticity**. Earlier we **created** the token with `sign`; now we **verify** it with `verify` from the same `jsonwebtoken` module.

```
import jwt from "jsonwebtoken";
```

The arguments

```
jwt.verify(token, "hello1234", (error, payload) => {
  // ...
```

```
});
```

Argument	Meaning
1st	The **token** read from the Authorization header
2nd	The **secret key** — exactly the same one used in `sign` (`hello1234`)
3rd	A **callback function**, called once verification is done

The lecturer checked the package documentation on **npmjs.com** (search any package, see its install instructions and function examples such as `jwt.sign` and `jwt.verify`) to confirm the argument order: token, secret key, callback.

The callback

Outcome	What the callback receives
Verification **fails** (token invalid, doesn't match the secret, etc.)	an **error**
Verification **succeeds**	the **payload** of the token — the same payload created at generation time (our `adminId`)

Decision:

- `error` present → the header existed but the token was not valid → send **401** with *token is not valid*.
- no error → the token is genuine → call `next()` so the next function (the controller) runs.

Complete middleware

File: `src/middlewares/verifyToken.js`

```
import jwt from "jsonwebtoken";

export function verifyToken(request, response, next) {
  try {
    const authHeader = request.get("authorization");

    if (authHeader) {
      const token = authHeader.split(" ")[1];

      jwt.verify(token, "hello1234", (error, payload) => {
        if (error) {
```

```
          response.status(401).send({ message: "token is not valid" });
        } else {
          next();
        }
      });
    } else {
      response.status(401).send({ message: "token is missing, please log in first" });
    }
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
}
```

Order of operations (recap)

1. Read the Authorization header.
2. If it exists, extract the token.
3. Verify the token with the secret key.
4. If there is an error → token invalid → 401.
5. If no error → `next()` → token valid.
6. If there was no header at all → `else` → "token is missing".

Background: The callback receives `payload` — you could store it on the request (for example `request.adminId = payload.adminId`) so the controllers know who is calling. The lecture only calls `next()`. A header such as `Authorization: Bearer` with no token after the space yields `token === undefined`; `jwt.verify` reports an error, so the request ends in the "token is not valid" branch.

Where does the secret live in real apps?

A student reacted that remembering the secret key would be hard. The lecturer: in real applications nobody remembers it — you **fetch it from a `.env` file or from some service**. As a beginner you simply remember: the key used to encrypt at login **must be the same key at verification**.

> **Exam:** `jwt.verify(token, secret, callback)`. Callback gets `(error, payload)`. Error → 401 "token is not valid". No error → `next()`. Same secret as `sign`.

Guarding the Product Routes and End-to-End Test

Putting the middleware on the routes

In `ProductRouter.js`, import `verifyToken` from `../middlewares/verifyToken.js` and put it **before the controller** on every route to be secured:

```
import { Router } from "express";
import { verifyToken } from "../middlewares/verifyToken.js";
import {
  save,
  getAll,
  getById,
  deleteById,
  updateById,
} from "../controllers/ProductController.js";

const productRouter = Router();

productRouter.post("/product", verifyToken, save);
productRouter.get("/product", verifyToken, getAll);
productRouter.get("/product/:id", verifyToken, getById);
productRouter.delete("/product/:id", verifyToken, deleteById);
productRouter.put("/product/:id", verifyToken, updateById);

export default productRouter;
```

The `admin` routes (`POST /admin`, `POST /admin/login`) do **not** get this middleware. Restart the server after changes.

The lecturer's end-to-end test (GET all products)

#	Request	Expected	Why
1	Login: **POST** `/admin/login`, body `{ "phone": "808080", "password": "Suresh123" }`	login successful + `token`	Get the proof. Copy the token.
2	**GET** `/product`, **no** Authorization header	`token is missing`	Header absent
3	**GET** `/product`, `Authorization: <random text>`	`token is not valid`	Header exists, token fails verification
4	**GET** `/product`, `Authorization: Bearer <real token>`	the products array	Valid token → `next()` → `getAll`

During the live run he used the Postman quirk of opening a new request tab when a request did not behave; the lesson was the 4-step flow, not the tab.

Result: all product APIs — save, get all, get by id, delete by id, and update — are now secured with JWT authentication.

> **Exam:** Place `verifyToken` before the controller on each secured route. Test missing, invalid, and valid token to prove the three paths.

Beyond Basic Authentication

The lecturer repeated that what the class built is "not even a hello world of real-time applications — it is just a lowercase *h*". What this simple version is missing:

Missing piece	Why it matters
Token expiry	Without one, a stolen token works forever: "if it falls into anyone's hand, they get access to your system"
Private / public keys	Replaces the hard-coded secret
Role-based access	Store each user's role in the DB; generate the token based on role; limit actions per role
Access + refresh tokens	Two kinds of tokens (below)

Access token vs refresh token

- At login the user receives an **access token** and a **refresh token**.
- The **access token** is used to **call APIs**; it is short-lived.
- The **refresh token** is used to **refresh** the access token — a service running in the background, say every 15–20 minutes, expires the old token and sets a new one.

His answer to a student's question: on a website that never seems to log you out, behind the scenes you are repeatedly logged out and in. *One token never lives for a long time*; a service keeps generating new tokens and expiring old ones in the background. That is possible with refresh tokens.

Domain names to remember

- **Authentication and RBAC** — "Authentication and Role-Based Access Control" — a huge domain and a "very major concern" for any application.
- **OAuth** — "you people cannot even write basic code, so leave authentication to OAuth": it can handle login, token authentication, and RBAC for you.
- **Clerk** — an implementation (service) that can handle the entire authentication of an application.

Final message: authentication is a very sensitive concern for enterprise apps; once someone gets unauthorised access and you do not notice, much damage is done. Take it seriously, learn basic authentication properly first, step by step, and accept that your code will crash many times along the way ("even I faced difficulties; there was no Abhishek sir at my time to ask").

> **Exam:** Missing in the basic version: expiry, key pairs, roles. Real apps: access token (call APIs) + refresh token (renew access token), RBAC, OAuth / services like Clerk.

Combined Worked Example: Secured CRUD Server

One copy-paste-runnable file that ties together register, hashing, login, JWT, header, and middleware. (The lecture's project splits this into controllers / routers / middlewares; this single file keeps the logic visible in one place.)

Setup

```
CREATE DATABASE IF NOT EXISTS productdb;
USE productdb;

CREATE TABLE IF NOT EXISTS product (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(200),
  price FLOAT,
  description VARCHAR(300),
  quantity INT
);

-- admin table: id, name, phone, password (widened to 1000 as in the lecture)
CREATE TABLE IF NOT EXISTS admin (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100),
  phone VARCHAR(20),
```

```
  password VARCHAR(1000)
);
```

```
npm init -y
npm install express mysql2 bcrypt jsonwebtoken
```

Add `"type": "module"` to `package.json`. Change the MySQL user/password below to yours.

File: `index.js`

```
import express from "express";
import { createPool } from "mysql2/promise";
import { hashSync, compareSync } from "bcrypt";
import jwt from "jsonwebtoken";

const app = express();
app.use(express.json());

// Practice only: real apps keep keys in the deployment environment.
const SECRET_KEY = "hello1234";

const sqlConnectionPool = createPool({
  host: "localhost",
  port: 3306,
  user: "root",
  password: "root",
  database: "productdb",
  connectionLimit: 10,
  queueLimit: 0,
});

// ———————— MIDDLEWARE: runs before every secured controller ————————
function verifyToken(request, response, next) {
  try {
    const authHeader = request.get("authorization");   // "Bearer <token>"

    if (authHeader) {
      const token = authHeader.split(" ")[1];           // index 1 = the JWT

      jwt.verify(token, SECRET_KEY, (error, payload) => {
        if (error) {
          response.status(401).send({ message: "token is not valid" });
        } else {
          next();                                       // genuine token
        }
      });
    } else {
      response.status(401).send({ message: "token is missing, please log in first" });
    }
```

```
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
}

// ———————— OPEN: register an admin ————————
app.post("/admin", async (request, response) => {
  try {
    const { name, phone, password } = request.body;
    const encryptedPassword = hashSync(password, 10);   // never store plain text

    const insertQuery =
      "INSERT INTO admin (name, phone, password) VALUES (?, ?, ?)";
    await sqlConnectionPool.query(insertQuery, [name, phone, encryptedPassword]);

    response.send({ message: "admin got registered" });
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
});

// ———————— OPEN: login, returns a JWT ————————
app.post("/admin/login", async (request, response) => {
  try {
    const { phone, password } = request.body;

    const loginQuery = "SELECT * FROM admin WHERE phone = ?";
    const [result] = await sqlConnectionPool.query(loginQuery, [phone]);

    if (result.length === 0) {
      response.status(401).send({ message: "phone number is not valid" });
      return;
    }

    const adminData = result[0];
    const isPasswordValid = compareSync(password, adminData.password);

    if (isPasswordValid) {
      const token = jwt.sign({ adminId: adminData.id }, SECRET_KEY);
      response.send({ message: "login successful", token });
    } else {
      response.status(401).send({ message: "password is not valid" });
    }
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
});
```

```
// ———————— SECURED: product APIs (verifyToken before the controller) ————————
app.post("/product", verifyToken, async (request, response) => {
  try {
    const { name, price, description, quantity } = request.body;
    const insertQuery =
      "INSERT INTO product (name, price, description, quantity) VALUES (?, ?, ?, ?)";
    const [result] = await sqlConnectionPool.query(insertQuery, [
      name,
      price,
      description,
      quantity,
    ]);
    response.send({ message: "product inserted", insertId: result.insertId });
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
});

app.get("/product", verifyToken, async (request, response) => {
  try {
    const [result] = await sqlConnectionPool.query("SELECT * FROM product");
    response.send(result);
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
});

app.get("/product/:id", verifyToken, async (request, response) => {
  try {
    const { id } = request.params;
    const [result] = await sqlConnectionPool.query(
      "SELECT * FROM product WHERE id = ?",
      [id]
    );
    if (result.length > 0) {
      response.send(result[0]);
    } else {
      response.status(404).send({ message: `product with ID ${id} not found` });
    }
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
});

app.put("/product/:id", verifyToken, async (request, response) => {
  try {
    const { id } = request.params;
    const { name, price, description, quantity } = request.body;
```

```
    const updateQuery =
      "UPDATE product SET name = ?, price = ?, description = ?, quantity = ? WHERE id
= ?";
    const [result] = await sqlConnectionPool.query(updateQuery, [
      name,
      price,
      description,
      quantity,
      id,
    ]);
    if (result.affectedRows === 0) {
      response.status(404).send({ message: `product with ID ${id} not found` });
      return;
    }
    response.send({ message: "product updated" });
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
});

app.delete("/product/:id", verifyToken, async (request, response) => {
  try {
    const { id } = request.params;
    const [result] = await sqlConnectionPool.query(
      "DELETE FROM product WHERE id = ?",
      [id]
    );
    if (result.affectedRows === 0) {
      response.status(404).send({ message: `product with ID ${id} not found` });
      return;
    }
    response.send({ message: "product deleted" });
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "something went wrong" });
  }
});

app.listen(6500, () => {
  console.log("Secured CRUD APIs listening on 6500");
});
```

End-to-end trace

1. `node index.js` — pool ready, server on port 6500.
2. **Register:** `POST http://localhost:6500/admin` with body

```
{
  "name": "Suresh",
  "phone": "808080",
  "password": "Suresh123"
}
```

`hashSync("Suresh123", 10)` → a long hash; the INSERT stores the **hash**. Response: *admin got registered*. `SELECT * FROM admin` shows the hash, not `Suresh123`.

3. **Login with a wrong phone** (e.g. `111111`): the SELECT returns `[]` → 401 *phone number is not valid*.
4. **Login with right phone, wrong password:** row found, `compareSync` returns `false` → 401 *password is not valid*.
5. **Login correctly:** `POST /admin/login` with `{ "phone": "808080", "password": "Suresh123" }` → `compareSync` is `true` → `jwt.sign({ adminId: 1 }, "hello1234")` → response `{ "message": "login successful", "token": "eyJ ... " }`. Copy the token.
6. **Call without a token:** `GET /product` → `verifyToken` finds no header → 401 *token is missing, please log in first*. The controller never runs.
7. **Call with a bad token:** header `Authorization: Bearer abc` → `jwt.verify` fails → 401 *token is not valid*.
8. **Call with the real token:** header `Authorization: Bearer eyJ ...` → `split(" ")[1]` yields the JWT → verification succeeds → `next()` → `SELECT * FROM product` → the array of products.
9. `POST /product`, `DELETE /product/2`, etc. behave the same way: no token → blocked, valid token → controller runs.

Quick Revision Checklist

Topic	One-line takeaway
Open API problem	Anyone with the URL could create/update/delete products; only a logged-in admin should.
Flow	Register → login → server issues token → client sends token on every secured call → middleware verifies.
Open vs secured	Register and login stay open; product APIs are secured.

Topic	One-line takeaway
Plain-text passwords	Never store them; hash with a one-way function so they cannot be decrypted back.
bcrypt	Third-party module: plain password in, hashed string out; `npm install bcrypt`.
hash vs hashSync	`hash` async, `hashSync` sync; choose sync so INSERT waits for the hash.
hashSync args	`(plainPassword, rounds)` e.g. `hashSync(password, 10)`; returns a string.
Same password	Two users with identical passwords get completely different hashes.
Password column	`ALTER TABLE admin MODIFY password VARCHAR(1000)` so the hash never truncates.
Login comparison	SQL compares phone only; bcrypt compares password (plain vs hash).
compareSync	`compareSync(plain, encrypted)` returns a boolean; `compare` is the async twin.
Empty result	`result.length === 0` → no admin with that phone → early "phone number is not valid".
JWT	JSON Web Token: header . payload . signature, three dot-separated parts.
Payload	Your JSON (admin id, login time, expiry, role); the only part you specify.
Signature + secret	Proves the token was created with the server's secret; stops fake/duplicate tokens.
Secret key	Practice: hard-coded string; real: RSA private/public key, set in deployment, rotated.
jsonwebtoken	`npm i jsonwebtoken`; `jwt.sign(payload, secret)` creates, `jwt.verify(token, secret, cb)` checks.
Token location	Header only: not URL (exposed), not body (no body in GET; keep body for business data).
Authorization header	Key `Authorization`, value `Bearer <space><token>`; Bearer is the type for JWT.
Reading headers	`request.get("authorization")`; token = `authHeader.split(" ")[1]`.
401	Missing/invalid token → 401 Unauthorized (client's fault, 4xx).
verifyToken	Header missing → 401; verify error → 401; success → `next()`.
Route order	`router.get("/product", verifyToken, getAll)` — middleware before controller.
Beyond basics	Expiry, key pairs, RBAC, access + refresh tokens, OAuth / Clerk.

Practice Exercises

Concept

1. Name the three jobs involved in "only the admin may manage products" and which API handles each.
2. Why are the register and login APIs deliberately **not** secured?
3. Why does the lecturer insist that an encrypted password must not be reversible? What goes wrong if anyone can decrypt it?
4. Why do `hashSync` and `compareSync` suit this lecture better than `hash` and `compare`?
5. Two users both register with the password `Test123`. Will their stored values be equal? Why does that matter for login?
6. What do the three parts of a JWT represent, and which one does the developer actually write?
7. Using the restaurant analogy, explain what the "secret ink" and the "torchlight" correspond to in a JWT system.
8. Give two reasons the token cannot travel in the request body, and one reason it cannot travel in the URL.

Application

9. Write the `ALTER TABLE` statement that widens the `password` column to 1000 characters.
10. Write the `loginQuery` and `query` call that fetch the admin row by phone, without risking SQL injection.
11. Write the one line that creates a JWT whose payload carries the admin's id.
12. Show the exact `Authorization` header you would enter in Postman for a token `abc.def.ghi`.
13. Write the two lines of code that read the Authorization header and extract the token from it.
14. In which order do `verifyToken` and the controller appear in `productRouter.delete( ... )`?

Predict output

15. A client sends `GET /product` with header `Authorization: whatever` when the middleware only checks that the header exists. What happens? What changes once `jwt.verify` is added?
16. `SELECT * FROM admin WHERE phone = ?` finds no row. What is `result`, what is `result.length`, and which message is sent?
17. `"Bearer eyJabc".split(" ")` — what is the returned array, and what is `[1]`?

Spot the bug

18. Find the bugs in this middleware:

```
export function verifyToken(request, response, next) {
  const authHeader = request.get("authorization");
  const token = authHeader.split(" ")[1];
  jwt.verify(token, "hello123", (error, payload) => {
    if (error) {
      response.status(401).send({ message: "token is not valid" });
    }
    next();
  });
}
```

19. Find the bugs in this register code:

```
export async function save(request, response) {
  const { name, phone, password } = request.body;
  const encryptedPassword = hashSync(password, 10);
  const insertQuery = `INSERT INTO admin (name, phone, password)
VALUES ('${name}', '${phone}', '${password}')`;
  await sqlConnectionPool.query(insertQuery);
  response.send({ message: "admin got registered" });
}
```

20. A developer writes the login check as `SELECT * FROM admin WHERE phone = ? AND password = ?` with `[phone, password]` where `password` is the plain text from the body. Why does login always fail, and what is the fix?

Practice Answers

1. Registration (who may exist as an admin) → `POST /admin`. Authentication/login (is this person that admin; issue a token) → `POST /admin/login`. Access control (is this request logged in) → the `verifyToken` middleware on the product APIs.

2. A caller cannot log in without first registering and cannot get a token without calling login. If login required a token, nobody could ever obtain one. Only APIs handling protected resources (products) are secured.

3. If anyone could decrypt it, hiding it would be pointless — "Mithun hiding behind a bicycle from bullets". Anyone with database access could read every user's password. With a one-way hash, the original cannot be recovered; the system only ever needs to *compare*.

4. In the register and login controllers the next line depends on the result (INSERT needs the hash; the `if` needs the boolean). The synchronous variants make the code wait for the result, so the lecture uses `hashSync` and `compareSync`. (The async variants are fine when awaited and do not block the server.)

5. No — they will be **totally different**, because each hash includes randomness (a salt). So you cannot compare passwords with `=` in SQL; only bcrypt's `compare` can tell whether a plain password matches a given hash.

6. Header = metadata such as the algorithm used; payload = your JSON data (admin id, login time, expiry, role); signature = tied to the secret key and proves authenticity. The developer writes only the **payload** (and supplies the secret); header and signature are produced automatically.

7. The secret ink is the **secret key** embedded through the signature; the torchlight is `jwt.verify` checking the token with that key. A duplicate token looks identical but lacks a valid signature, so verification fails.

8. Body: (1) GET requests have no body, so GET APIs could not be authenticated; (2) the body should only hold business-entity data (separation of concerns), not token or language. URL: it is sensitive data and anything in the URL is exposed to the whole internet.

9.

```
ALTER TABLE admin MODIFY password VARCHAR(1000);
```

10.

```
const loginQuery = "SELECT * FROM admin WHERE phone = ?";
const [result] = await sqlConnectionPool.query(loginQuery, [phone]);
```

11.

```
const token = jwt.sign({ adminId: adminData.id }, "hello1234");
```

12. Key `Authorization`, value:

```
Bearer abc.def.ghi
```

13.

```
const authHeader = request.get("authorization");
const token = authHeader.split(" ")[1];
```

14. `verifyToken` first, then the controller: `productRouter.delete("/product/:id", verifyToken, deleteById);`

15. With only the existence check, the request passes (a header exists) and the products are returned — nothing validated the token. After adding `jwt.verify`, the random value fails verification, the callback receives an error, and the response is **401 "token is not valid"**.

16. `result` is `[]` (a blank array), `result.length` is `0`, and the early response "phone number is not valid" is sent (e.g. with status 401).

17. `["Bearer", "eyJabc"]`; `[1]` is `"eyJabc"`, the token.

18.

- No try/catch around the middleware; if the header is missing `authHeader.split` throws (`undefined`).
- No "header missing" branch with a 401 response.
- Secret `"hello123"` does not match the secret used in `sign` (`hello1234`), so every real token fails.
- After sending the 401 in the error branch, the code still calls `next()` — the controller would also run, and the response would be sent twice. `next()` belongs in an `else`.

19.

- The query concatenates `${name}`, `${phone}` and the **plain** `${password}` into the SQL — SQL injection risk, and the plain-text password is stored.
- `encryptedPassword` is computed but never used. The values array should be `[name, phone, encryptedPassword]` with `?` placeholders.
- There is no try/catch, so a failure would leave the client without a proper error response (500).

20. The database holds the **encrypted** password; the request carries the **plain** one; they can never be equal, so the `AND password = ?` condition never matches. Fix: `SELECT * FROM admin WHERE phone = ?`, then `compareSync(password, adminData.password)` in JavaScript.